

UNIVERSITA' DEGLI STUDI DI BOLOGNA - CORSO DI LAUREA IN INFORMATICA
PROVA SCRITTA DI SISTEMI OPERATIVI
ANNO ACCADEMICO 2024/2025
06 febbraio 2026

Esercizio -1: Essere iscritti su AlmaEsami per svolgere questa prova.

Esercizio 0: Scrivere correttamente nome, cognome, matricola e posizione in tutti i fogli prima di svolgere ogni altro esercizio. Scrivere esclusivamente a penna senza abrasioni. E' vietato l'uso delle penne cancellabili, della matita, dei coprenti bianchi per la correzione (bianchetto) e la scrittura in colore rosso (riservato alla correzione). Il compito è formato da tre fogli, sei facciate compresa questa. Le soluzioni che si vogliono sottoporre per la correzione devono essere scritte negli spazi bianchi di questi fogli. Non verranno corretti altri supporti. E' obbligatorio consegnare il compito, è possibile chiedere che esso non venga valutato scrivendo "NON VALUTARE" in modo ben visibile nella prima facciata. Per svolgere questo compito occorre solo una penna e un documento di identità valido. La consultazione o anche solo la disponibilità di altro materiale comporterà l'annullamento del compito (verrà automaticamente valutato gravemente insufficiente).

Esercizio c.1: Scrivere un monitor **rbb** che implementi un buffer limitato con possibilità di ridimensionamento del buffer. Il monitor ha tre procedure entry:
__init__(unsigned init size)
procedure entry Obj read()
procedure entry void write(Obj val)
procedure entry void resize(unsigned int size)
La funzione **resize** modifica l'ampiezza del buffer: se l'ampiezza viene aumentata devono essere sbloccate le eventuali richieste **write** in sospeso fino al riempimento della nuova ampiezza. Se l'ampiezza viene diminuita vengono cancellati dal buffer gli elementi in eccesso più recentemente inseriti.

Esercizio c.2: Scrivere usando i semafori le due funzioni seguenti:
void delay(int ntick)
void tick(void)
la funzione **delay** è sempre bloccante; il parametro **ntick** (che deve essere positivo) indica il numero delle successive chiamate **tick** necessarie per riattivare il processo che ha chiamato **delay**,
Esempio. Il processo **P** chiama **delay(3)** e si blocca. Alla prima chiamata **tick()** **P** rimane bloccato così come alla seconda, alla terza **tick** **P** viene riattivato. La **syndelay** deve gestire chiamate **delay** da molteplici processi.
Esempio consideriamo la sequenza: **P** chiama **delay(3)**; **tick()**, **Q** chiama **delay(2)**, **tick()**. Il successivo **tick** riattiva sia **P** sia **Q**.

Esercizio g.1: Sia data la seguente stringa di riferimenti e la conseguente evoluzione dello stato della memoria (di 4 frame: f0 f1 f2 f3).

	0	1	2	3	0	1	2	4	0	1	2	4	3	1	2	4	5	4	2	1	4	5	3	4	5	1
	x	x	x	x				x					x				x						x			
f0	0	0	0	0	0	0	0	0	0	0	0	0	3	3	3	3	5	5	5	5	5	5	5	5	5	5
f1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
f2		2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	3	3	3	3
f3			3	3	3	3	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4

Questa situazione si può verificare usando un algoritmo di rimpiazzamento MIN? Perché?
Questa situazione si può verificare usando un algoritmo di rimpiazzamento LRU? Perché?
Questa situazione si può verificare usando un algoritmo di rimpiazzamento FIFO? Perché?

Esercizio g.2: rispondere alle seguenti domande (*motivando opportunamente le risposte!*):

- a) In una tabella FAT possono esistere elementi di uguale valore? (non considerando l'elemento nullo che contraddistingue la fine del file) Se sì: in quali casi? Se no: quali effetti avrebbe?
- b) Trovare, se possibile, un grafo di holt che abbia 5 nodi, tutte le risorse abbiano molteplicità due (due unità per ogni classe di risorse) e tutti i processi siano in deadlock.
- c) Perché i metodi di sincronizzazione tipo test&set (detti anche spinlock) assumono grande rilevanza nella scrittura di sistemi operativi multiprocessore?
- d) Perché è difficile revocare una autorizzazione fornita tramite capability?